MLDL Practical 10

Name: Ishaan Khan
Class: D15C
Roll No: 29
Batch: B

Aim: Implement a Convolutional Denoising Autoencoder for image reconstruction on the MNIST Digits dataset

Dataset Source

Dataset Name: MNIST Handwritten Digits Dataset
Platform: TensorFlow Keras (built-in)
Dataset Link: tensorflow.keras.datasets.mnist

Dataset Description

The MNIST dataset is a foundational benchmark in computer vision, consisting of grayscale images of handwritten digits (0–9). It is used here in an unsupervised setting — labels are discarded and the autoencoder is trained solely to reconstruct clean digit images from noisy inputs.

Property	Details
Total Samples	70,000 images (60,000 training + 10,000 testing)
Image Type	Grayscale handwritten digits (0–9)
Resolution	28 × 28 pixels (1 channel)
Number of Classes	10 digit classes (not used — unsupervised task)
Dataset Source	tensorflow.keras.datasets (built-in)
Noise Type	Gaussian noise with factor 0.3, clipped to [0, 1]

Column Name	Data Type	Description
Image	Float32 (28×28 matrix)	Normalized grayscale pixel values in range [0, 1]
Label	Integer (0–9)	Digit category — not used in this unsupervised experiment

Theoretical Framework

Autoencoders are unsupervised deep learning models that learn efficient data representations by compressing input into a lower-dimensional latent space (encoding) and then reconstructing it back to the original dimensions (decoding). The model is trained using clean images as both the target output, learning to map noisy inputs to clean outputs.

In a Denoising Autoencoder, the input is first corrupted with noise:

$$\acute{y} = x + \varepsilon, \quad \text{where } \varepsilon \sim N(0, \sigma^2)$$

The model learns a mapping $f(\acute{y}) \approx x$, reconstructing the original clean image from the corrupted input. This forces the encoder to learn robust latent representations rather than simply memorizing pixel values.

Encoder

The encoder compresses the 28×28 input image into a compact latent representation using convolutional layers followed by max pooling. Each Conv2D layer extracts spatial features, while MaxPooling2D reduces the spatial dimensions by half, creating a progressively abstract representation.

Decoder

The decoder mirrors the encoder structure using Conv2D layers and UpSampling2D operations to progressively restore the spatial resolution back to 28×28. A final Conv2D layer with sigmoid activation produces pixel values in [0, 1].

Loss Function – Binary Cross-Entropy

Binary Cross-Entropy is used as the reconstruction loss since pixel values are normalized to [0, 1]:

$$L = -(1/N) \Sigma [x_i \log(\hat{y}_i) + (1-x_i) \log(1-\hat{y}_i)]$$

Where x_i is the original clean pixel value and $\hat{y}_i$ is the reconstructed pixel value. This loss penalizes the model for deviating from the clean target and is minimized via backpropagation.

Algorithm Limitations

- **Smoothing Artifacts:** Pixel-wise Binary Cross-Entropy can cause over-smoothing in reconstructed images, blurring sharp edges and fine digit strokes.
- **No Semantic Understanding:** The autoencoder learns pixel-level reconstruction without understanding the semantic meaning of digit shapes.
- **Fixed Noise Distribution:** The model is trained for a specific noise type (Gaussian, factor 0.3) and may generalize poorly to different noise patterns.
- **Latent Space Unstructured:** Unlike Variational Autoencoders (VAEs), the latent space is not regularized and cannot be used for generation or interpolation.

Methodology / Workflow

Step	Phase	Action
1	Data Loading	Load MNIST digit images from tensorflow.keras.datasets
2	Preprocessing	Normalize pixel values to [0, 1] and reshape to (28, 28, 1)
3	Noise Injection	Add Gaussian noise (factor = 0.3) and clip values to [0, 1]
4	Encoder Design	Stack Conv2D + MaxPooling2D layers to compress input to latent space
5	Decoder Design	Stack Conv2D + UpSampling2D layers to reconstruct image from latent space
6	Compilation	Use Adam optimizer (LR = 0.001) with Binary Cross-Entropy loss
7	Training	Train for 15 epochs with batch size 128 on noisy input → clean target pairs

8	Evaluation	Visualize original vs noisy vs denoised images and plot loss curves

Hyperparameter Tuning

The performance of the convolutional autoencoder is sensitive to architectural and optimization settings. The following hyperparameters were configured to achieve stable convergence and high-quality denoising:

Hyperparameter	**Value**	**Rationale**
Conv Filters (Layer 1)	32	Extracts low-level spatial features (edges, strokes) from digit images
Conv Filters (Layer 2)	16	Further compresses representation into a compact latent encoding
Kernel Size	(3, 3)	Standard receptive field suitable for 28×28 grayscale images
Pooling / Upsampling	(2, 2)	Halves spatial dims in encoder; restores them in decoder symmetrically
Noise Factor	0.3	Moderate corruption to challenge the model without making reconstruction impossible
Optimizer	Adam	Adaptive learning rate for efficient and stable convergence
Learning Rate	0.001	Standard Adam default ensuring smooth weight updates
Loss Function	Binary Cross-Entropy	Appropriate for pixel values normalized to [0, 1]
Epochs	15	Sufficient for loss plateau; further training yields diminishing returns
Batch Size	128	Efficient gradient estimation with manageable GPU memory usage

Performance Analysis

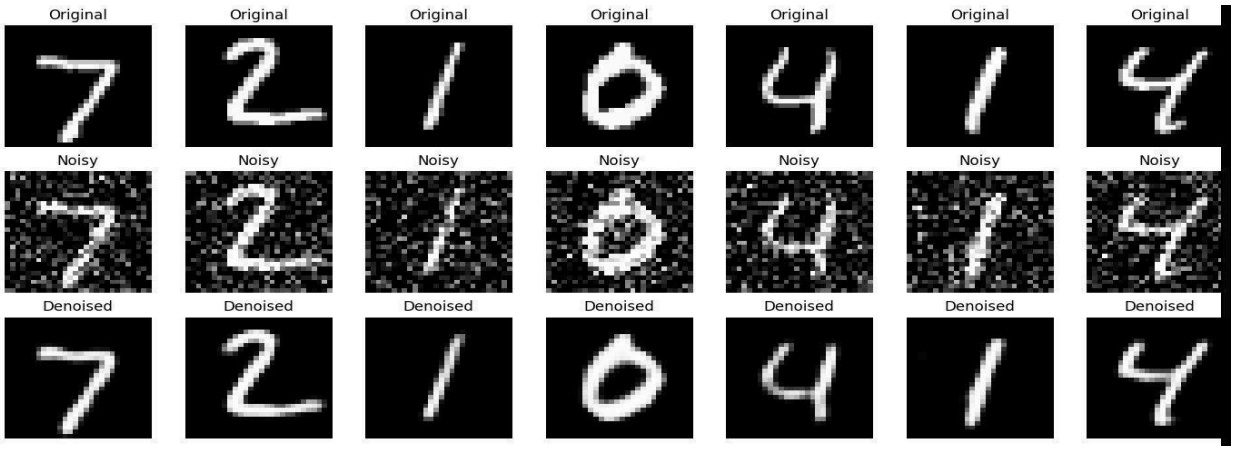
The Convolutional Denoising Autoencoder achieved stable convergence over 15 epochs, effectively learning to separate meaningful digit structure from random Gaussian noise.

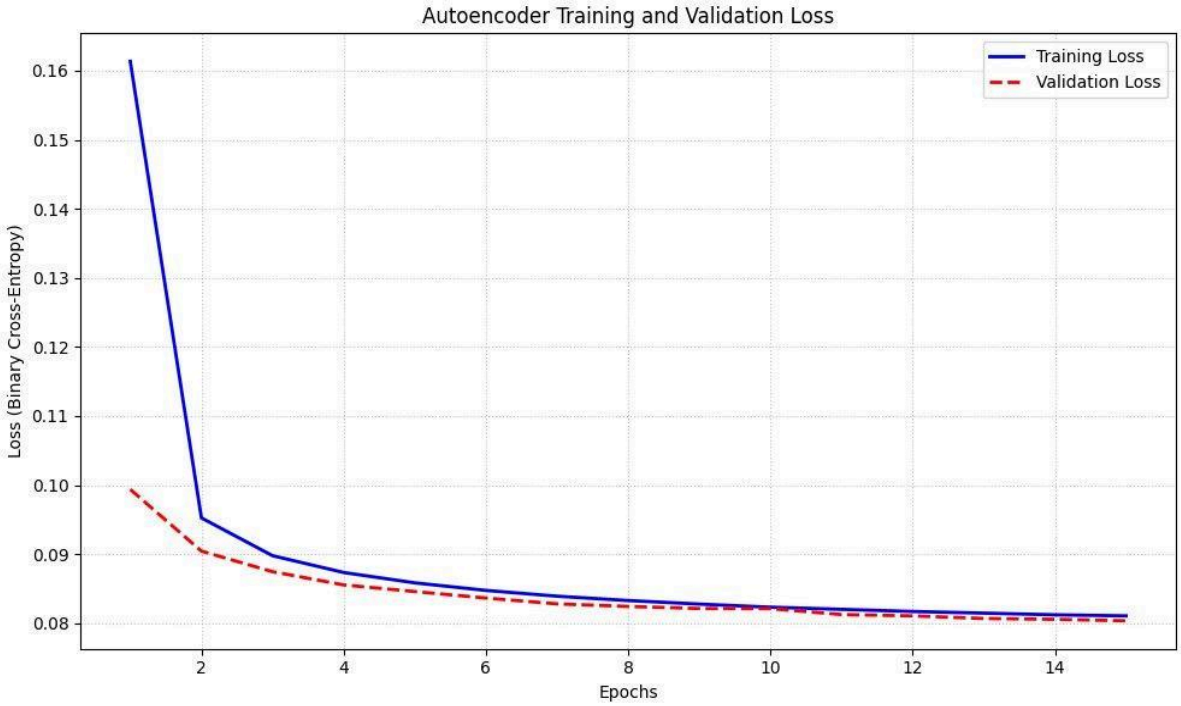
Metric	**Epoch 1**	**Epoch 15**
Training Loss	~0.1611	0.0811
Validation Loss	~0.0995	0.0803

- **Generalization:** Validation loss (0.0803) was slightly lower than training loss (0.0811) throughout training, indicating the model generalizes well to unseen noisy digit images without overfitting.

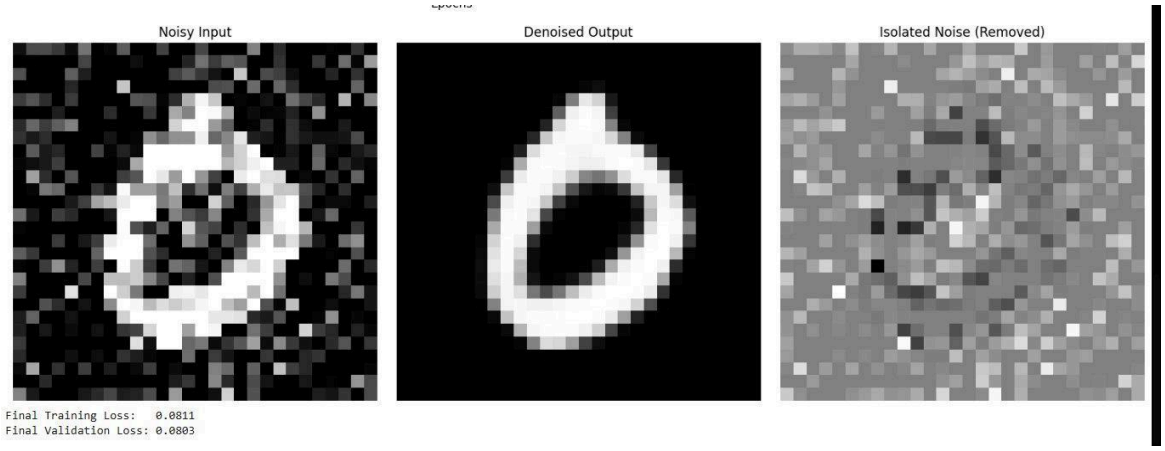
- **Rapid Early Convergence:** The loss drops sharply in the first 2–3 epochs, capturing the primary structural patterns of digits quickly, before settling into gradual refinement.
- **Reconstruction Quality:** Denoised outputs preserve essential digit structure — strokes, curves, and loops — with high fidelity, closely matching the original clean images.
- **Noise Isolation:** The isolated noise visualization confirms that the model specifically removes high-frequency random noise while retaining low-frequency structural signals.
- **Smoothing Effect:** Some mild blurring is visible in reconstructed digits due to the pixel-wise Binary Cross-Entropy loss, which is a known trade-off of this loss formulation.

Output





Final Training Loss: 0.0811
Final Validation Loss: 0.0803



Conclusion

In this experiment, a Convolutional Denoising Autoencoder was successfully implemented for noise removal on the MNIST handwritten digits dataset. With a final validation loss of 0.0803, the model demonstrated strong generalization and effective signal-noise separation.

The encoder-decoder architecture, combined with the Adam optimizer and Binary Cross-Entropy loss, enabled the model to learn robust latent representations that preserve digit structure while filtering out Gaussian noise. The loss curves confirm stable convergence with no overfitting across 15 epochs.

This experiment demonstrates how unsupervised convolutional architectures can be effectively applied to image restoration tasks, and highlights the potential of autoencoders as preprocessing components in larger computer vision pipelines.